

Name: - Shubham Das

S. A. P I. D: - 590033500

Date: - 05/05/2025

Programme: - B.TECH (CSE)

Assignment: - Prepare the algorithm and test case table. Then write, compile, execute, and test the C programme against the prepared use cases.

Write a menu-driven C programme using switch-cases to input a positive integer and perform one of the following operations based on the user's choice:

- 1. Find the sum and product of the first and last digits.**
- 2. Count how many digits are lesser than, equal, and greater than the given digit 'k'.**

Part 1: - Thorough understanding of the problem statement

Input: - 'num' (Positive integer to be processed, Variable type: 'int'. Unit: unitless integer)

'choice' (Menu option selection, Variable type: 'int', Unit: unitless integer)

'k' (Comparison digit for Option 2, Variable type: 'int', Unit: Unitless Integer, restricted to 0-9)

Output: - Option 1: 'sum' and 'product' of the first and last digits (Variable types: 'int')

Option 2: 'count_less', 'count_equal', 'count_greater', representing digit frequencies compared to 'k' (Variable types: int)

Formula: - Isolate last digit: 'last_digit = num % 10' ;

Isolate first digit: 'while (n >= 10) { n = n / 10; } first_digit = n' ;

Sum and Product: 'sum = first_digit; product = first_digit * last_digit' ;

Digit Extraction Loop (for counting): 'digit = n % 10' ;

followed by 'n = n / 10' ;

Part 2: - Writing and evaluating the algorithm

Step 1: - Start

Step 2: - Declare integer variables 'num', 'n', 'choice', 'first_digit', 'last_digit', 'sum', 'product', 'k', 'digit'.

Step 3: - Declare and initialize counters 'count_less = 0', 'count_equal = 0', 'count_greater = 0'.

Step 4: - Prompt the user to enter a positive integer and store it in 'num'.

Step 5: - Validate 'num'. If 'num <= 0', display an error and terminate.

Step 6: - Display the menu options (1: First/last digit math, 2: Compare digits to 'k').

Step 7: - Read the user's choice.

Step 8 (Case 1): - Set 'n = num'. Extract 'last_digit = n % 10'. Use a 'while (n >= 10)' loop dividing 'n' by 10 to find 'first_digit = n'. Calculate 'Sum = first_digit + last_digit'. Print the results. Break.

Step 9 (Case 2): - Prompt user for digit 'k' (0-9). Set 'n = num'. Use a 'while (n > 0)' loop: extract 'digit = n % 10'. Use 'if/else if/else' logic to increment 'count_less', 'count_equal', or 'count_greater' based on comparing 'digit' to 'k'. Update 'n = n / 10'. Print the results. Break.

Step 10 (Default): - Print an "Invalid choice" error message.

Step 11: - Stop

Part 3: - Writing and compiling the C programme

kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

1 2 3 4

kali@kali: ~

Session Actions Edit View Help

GNU nano 9.0 digit_ops.c

```
#include <stdio.h>

int main() {
    int num, n, choice, k, digit;
    int first_digit, last_digit, sum, product;
    int count_less = 0, count_equal = 0, count_greater = 0;

    // Input and Validation
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num <= 0) {
        printf("Error: Please enter a strictly positive integer greater than zero.\n");
        return 1;
    }

    // Menu Display
    printf("\n--- Digit Operation Menu ---\n");
    printf("1. Find sum and product of the first and last digits\n");
    printf("2. Count how many digits are lesser, equal, and greater than a given digit 'k'\n");
    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);

    printf("\n--- Result ---\n");
    switch (choice) {
        case 1:
            n = num;
            last_digit = n % 10;

            // Loop to find the first digit
            while (n >= 10) {
                n = n / 10;
            }
            first_digit = n;

            sum = first_digit + last_digit;
            product = first_digit * last_digit;

            printf("Number: %d\n", num);
            printf("First Digit: %d, Last Digit: %d\n", first_digit, last_digit);
            printf("Sum = %d, Product = %d\n", sum, product);
            break;

        case 2:
            printf("Enter the comparison digit 'k' (0-9): ");
            scanf("%d", &k);

            if (k < 0 || k > 9) {
```

[Read 81 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

GNU nano 9.0

```
sum = first_digit + last_digit;
product = first_digit * last_digit;

printf("Number: %d\n", num);
printf("First Digit: %d, Last Digit: %d\n", first_digit, last_digit);
printf("Sum = %d, Product = %d\n", sum, product);
break;

case 2:
printf("Enter the comparison digit 'k' (0-9): ");
scanf("%d", &k);

if (k < 0 || k > 9) {
printf("Error: 'k' must be a single digit between 0 and 9.\n");
return 1;
}

n = num;
while (n > 0) {
digit = n % 10;

if (digit < k) {
count_less++;
} else if (digit == k) {
count_equal++;
} else {
count_greater++;
}

n = n / 10; // Remove the last digit
}

printf("Number: %d, k: %d\n", num, k);
printf("Count_lesser = %d\n", count_less);
printf("Count_equal = %d\n", count_equal);
printf("Count_greater = %d\n", count_greater);
break;

default:
printf("Error: Invalid choice. Please select 1 or 2.\n");
return 1;
}

return 0;
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the terminal

Step 2: - Create the source code

Step 3: - Write and Save code

Step 4: - Compile the programme

Step 5: - Run the Binary Executable Programme

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

$ 1 2 3 4 $

Session Actions Edit View Help

(kali@kali)-[~]
$ nano digit_ops.c

(kali@kali)-[~]
$ gcc digit_ops.c -o digit_ops

(kali@kali)-[~]
$ ./digit_ops
Enter a positive integer: 12345

--- Digit Operation Menu ---
1. Find sum and product of the first and last digits
2. Count how many digits are lesser, equal, and greater than a given
   digit 'k'
Enter your choice (1 or 2): 1

--- Result ---
Number: 12345
First Digit: 1, Last Digit: 5
Sum = 6, Product = 5

(kali@kali)-[~]
$ ./digit_ops
Enter a positive integer: 12345

--- Digit Operation Menu ---
1. Find sum and product of the first and last digits
2. Count how many digits are lesser, equal, and greater than a given
   digit 'k'
Enter your choice (1 or 2): 2

--- Result ---
Enter the comparison digit 'k' (0-9): 3
Number: 12345, k: 3
Count_lesser = 2
Count_equal = 1
Count_greater = 2

(kali@kali)-[~]
$ ./digit_ops
Enter a positive integer: 909

--- Digit Operation Menu ---
1. Find sum and product of the first and last digits
2. Count how many digits are lesser, equal, and greater than a given
   digit 'k'
Enter your choice (1 or 2): 2

--- Result ---
Enter the comparison digit 'k' (0-9): 8
```

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

[Icons] | 1 2 3 4 | [Terminal Icon]

(kali@kali)~$ ./digit_ops
Enter your choice (1 or 2): 2

--- Result ---
Enter the comparison digit 'k' (0-9): 3
Number: 12345, k: 3
Count_lesser = 2
Count_equal = 1
Count_greater = 2

(kali@kali)~$ ./digit_ops
Enter a positive integer: 909

--- Digit Operation Menu ---
1. Find sum and product of the first and last digits
2. Count how many digits are lesser, equal, and greater than a given
   digit 'k'
Enter your choice (1 or 2): 2

--- Result ---
Enter the comparison digit 'k' (0-9): 8
Number: 909, k: 8
Count_lesser = 1
Count_equal = 0
Count_greater = 2

(kali@kali)~$ ./digit_ops
Enter a positive integer: 888

--- Digit Operation Menu ---
1. Find sum and product of the first and last digits
2. Count how many digits are lesser, equal, and greater than a given
   digit 'k'
Enter your choice (1 or 2): 2

--- Result ---
Enter the comparison digit 'k' (0-9): 8
Number: 888, k: 8
Count_lesser = 0
Count_equal = 3
Count_greater = 0

(kali@kali)~$ ./digit_ops
Enter a positive integer: -45
Error: Please enter a strictly positive integer greater than zero.

(kali@kali)~$
```

Part 5: - Standard test table

TEST CASE	INPUT (num, choice, k)	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT (PASS/FAIL)
1	num = 12345 choice = 1 k = N/A	Sum = 6 Product = 5	Sum = 6 Product = 5	PASS
2	num = 12345 choice = 2 k = 3	Lesser = 2 Equal = 1 Greater = 2	Lesser = 2 Equal = 1 Greater = 2	PASS
3	num = 909 choice = 1 k = 8	Sum = 18 Product = 81	Sum = 18 Product = 81	PASS
4	num = 888 choice = 2 k = 8	Lesser = 0 Equal = 3 Greater = 0	Lesser = 0 Equal = 3 Greater = 0	PASS
5	num = -45 choice = 1 k = N/A	Error: strictly positive	Error: strictly positive	PASS

Part 6: - Evaluation

The algorithm securely executes with a time complexity, where 'd' correlates to the number of digits inside 'num'.

